

Teaching Exam

Subject – Computer Science

Topic – DBMS



Lecture No.- 13

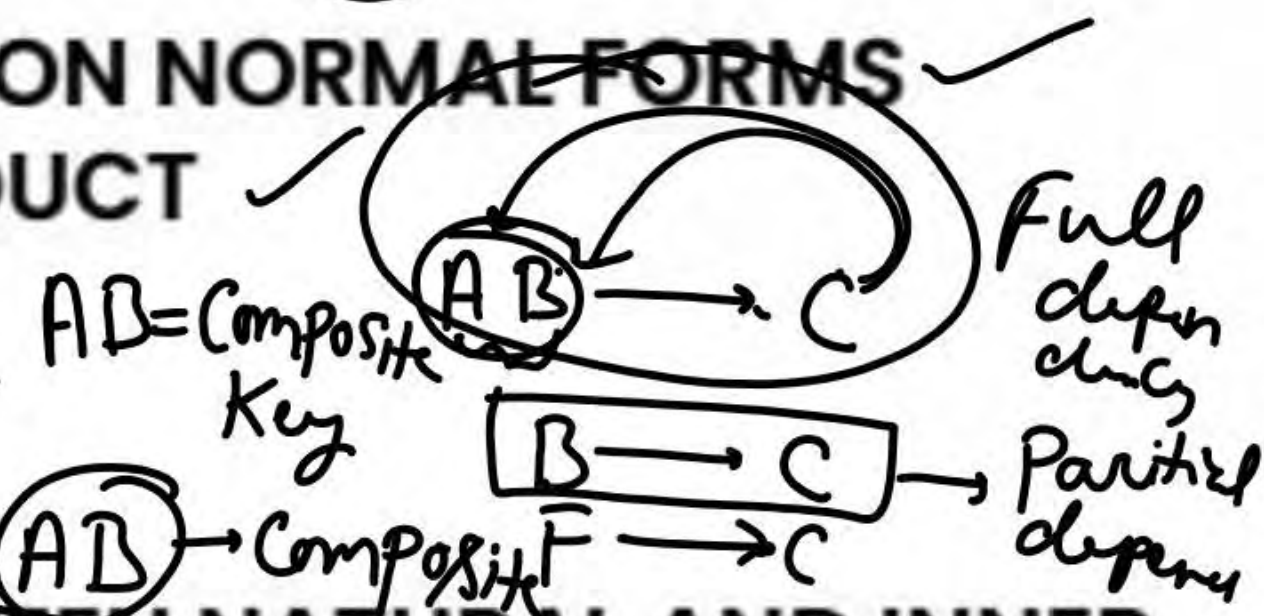
By- Rahul Sir

Topics to be Covered

Topic



- 1:- 3-NF ✓
- 2:- BCNF ✓
- 3:- SOME PRACTICE ON NORMAL FORMS ✓
- 4:- CRESTIAN PRODUCT ✓
- 5-JOINS ✓
- 6- NATURAL JOIN ✓
- 7- INNER JOIN ✓
- 8- DIFFERENCE BETWEEN NATURAL AND INNER JOIN
- 9->EQUI JOINS





Topic : 3-NF

What is Third Normal Form (3NF)?

A relation is in Third Normal Form (3NF) if it satisfies the following two conditions:

1. It is in **Second Normal Form (2NF)**: This means the table has no partial dependencies (i.e., no non-prime attribute is dependent on a part of a candidate key).
2. There is **no transitive dependency for non-prime attributes**: In simpler terms, no non-key attribute should depend on another non-key attribute. Instead, all non-key attributes should depend directly on the primary key.

Understanding Transitive Dependency

To fully grasp 3NF, it's essential to understand transitive dependency. A transitive dependency occurs when one non-prime attribute depends on another non-prime attribute rather than depending directly on the primary key. This can create redundancy and inconsistencies in the database.

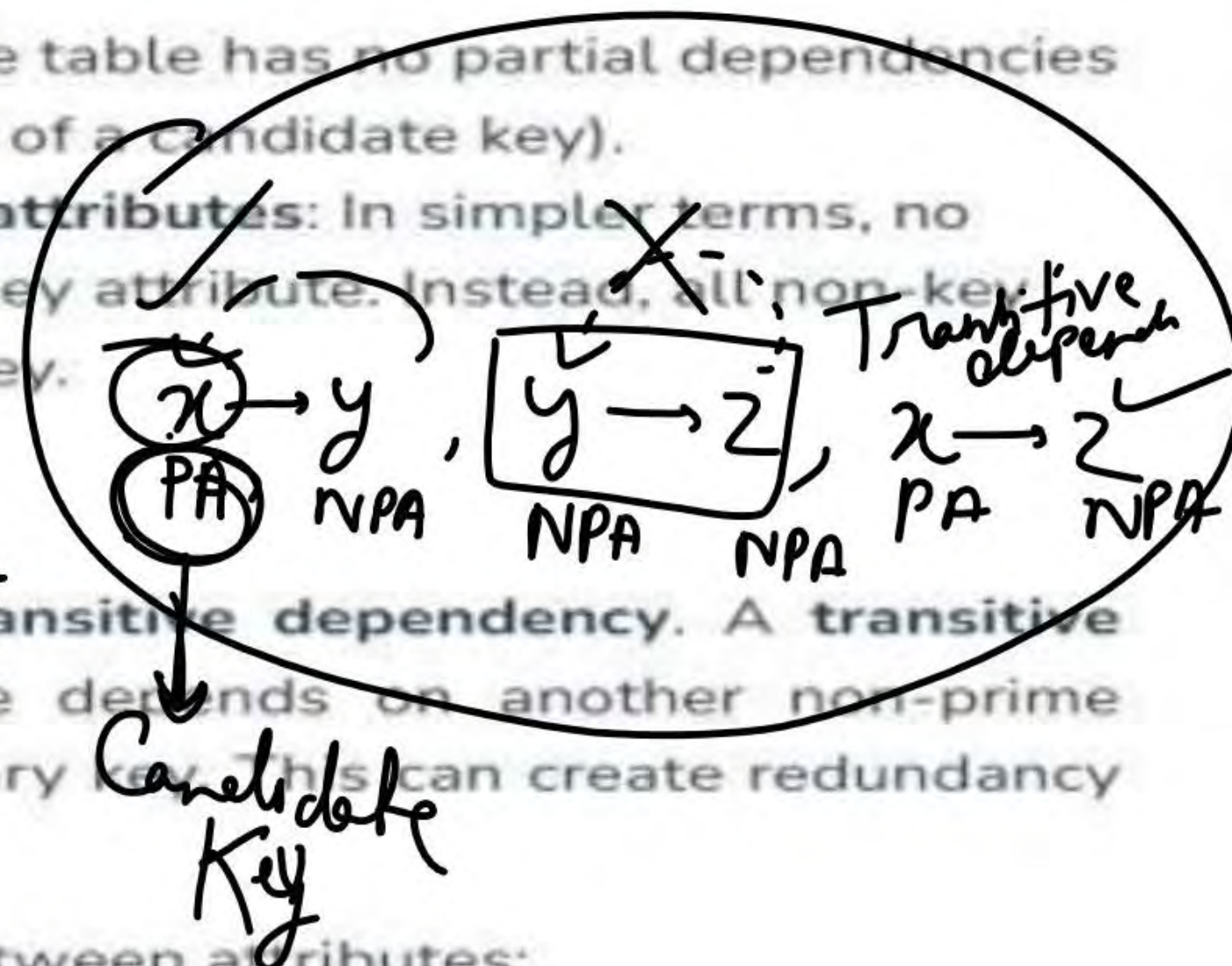
For example, if we have the following relationship between attributes:

- $A \rightarrow B$ (A determines B)
- $B \rightarrow C$ (B determines C)

$$\begin{aligned} x &\rightarrow y \\ x &\rightarrow z \end{aligned}$$



$$PA \rightarrow x, \quad (y, z) \rightarrow NPA$$

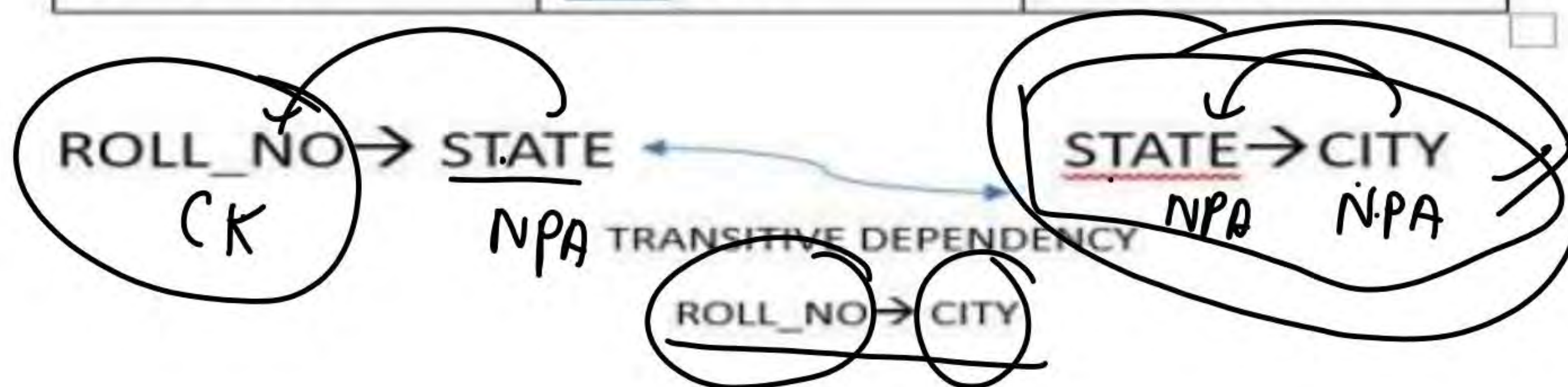




Topic : 3-NF

Roll no	State	city
1	U.P	MIRZAPUR
2	BIHAR	PATNA
3	U.P	MATHURA
4	PUNJAB	JALANDHAR
5	U.P	MATHURA

~~3NF~~





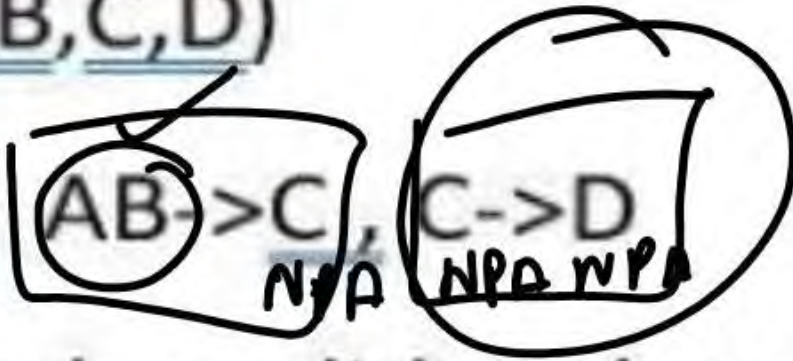
Topic : 3-NF



TEACHING
WALLAH

$R(\underline{A}, \underline{B}, \underline{C}, \underline{D})$

FD:-



$PA = \{A, B\}$
 $NPA = \{C, D\}$

I- Find candidate key

II- Find prime and non prime attribute

III- Check whether it is Transitive dependency or not

~~3NF~~ Yes



Topic : 3-NF

$R(\underline{A}, \underline{B}, \underline{C}, \underline{D})$

FD:- $AB \rightarrow CD$, $D \rightarrow A$ PD

I- Find candidate key

II- Find prime and non prime attribute

III- Check whether it is Transitive dependency or not

$B \rightarrow A$

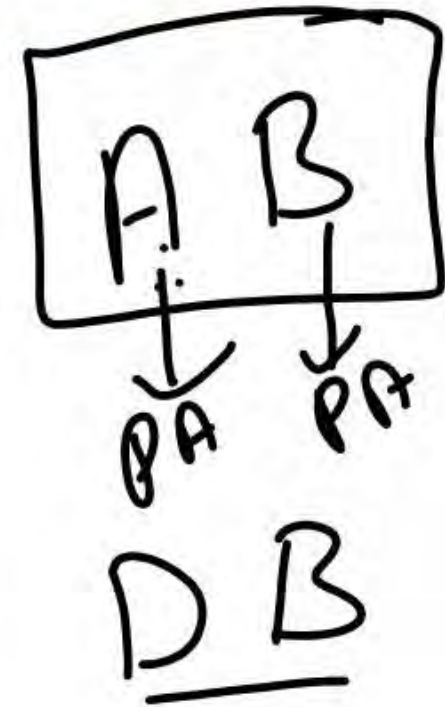
$\{AB, CD\}$

$\underline{AB}$

$\underline{DB}$

$A = \{ \bar{A} B \bar{D} \}$
 $\vee PA = \{ C \}$

2NF



$\times$
3NF

$\underline{AB}$
 $\underline{DB}$



Topic : 3-NF

$AB^+ = ABCD$

$DB^+ = DBAC$

Candidate key:- {AB, DB}

PRIME ATTRIBUTE={A, B, D}

NON PRIME ATTRIBUTE={C}

BCNF

RULE FOR CHECK 3-NF If relation is already follow 2-NF

= L.H.S Must be a Candidate key or super key

OR

~~R.H.S is a prime attribute~~

**Topic : Boyce-Codd Normal Form (BCNF)**

While Third Normal Form (3NF) is generally sufficient for organizing relational databases, it may not completely eliminate redundancy. Redundancy can still occur if there's a dependency $X \rightarrow X$ where X is not a candidate key. This issue is addressed by a stronger normal form known as **Boyce-Codd Normal Form (BCNF)**.

Applying the rules of 2NF and 3NF can help identify some redundancies caused by dependencies that violate candidate keys. However, even with these rules, certain dependencies may still lead to redundancy in 3NF. To overcome this limitation, BCNF was introduced by Codd in 1974 as a more robust solution.

**Topic : Boyce-Codd Normal Form (BCNF)**

While Third Normal Form (3NF) is generally sufficient for organizing relational databases, it may not completely eliminate redundancy. Redundancy can still occur if there's a dependency $X \rightarrow X$ where X is not a candidate key. This issue is addressed by a stronger normal form known as **Boyce-Codd Normal Form (BCNF)**.

Applying the rules of 2NF and 3NF can help identify some redundancies caused by dependencies that violate candidate keys. However, even with these rules, certain dependencies may still lead to redundancy in 3NF. To overcome this limitation, BCNF was introduced by Codd in 1974 as a more robust solution.



Topic : Boyce-Codd Normal Form (BCNF)

Boyce-Codd Normal Form (BCNF) is a stricter version of Third Normal Form (3NF) that ensures a more simplified and efficient database design. It enforces that every non-trivial functional dependency must have a superkey on its left-hand side. This approach addresses potential issues with candidate keys and ensures the database is free from redundancy.

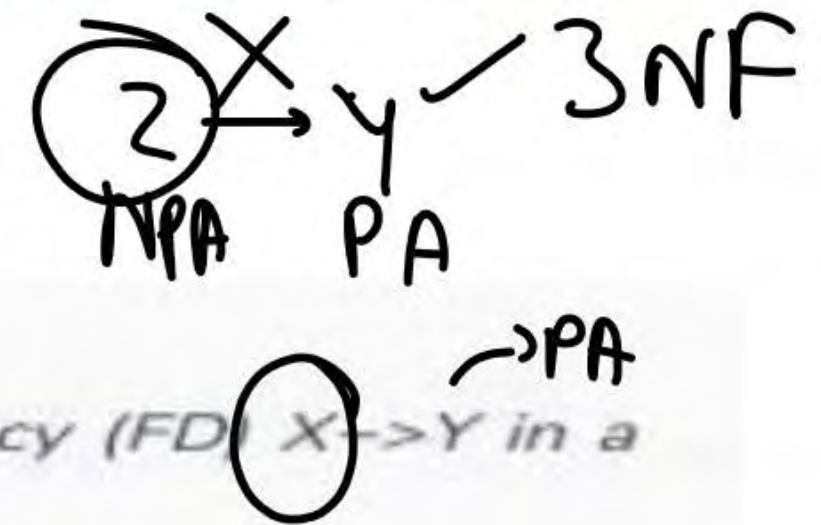
BCNF eliminates redundancy more effectively than 3NF by strictly requiring that all functional dependencies originate from super-keys.

BCNF is essential for good database schema design in higher-level systems where consistency and efficiency are important, particularly when there are many candidate keys (as one often finds with a delivery system).

Rules for BCNF

Rule 1: The table should be in the 3rd Normal Form.

Rule 2: X should be a super-key for every functional dependency (FD) $X \rightarrow Y$ in a given relation.



Note: To test whether a relation is in BCNF, we identify all the determinants and make sure that they are candidate keys.



Topic : Boyce-Codd Normal Form (BCNF)



ROLL_NO	NAME	VOTER_ID	AGE
1	RAM	M0123	20
2	HARI	K0234	26
3	RAM	A0235	24
4	SURESH	Z0256	23

Candidate key:- { ROLL_NO, VOTER_ID }

FD:- { ROLL_NO → NAME

ROLL_NO → VOTER_ID

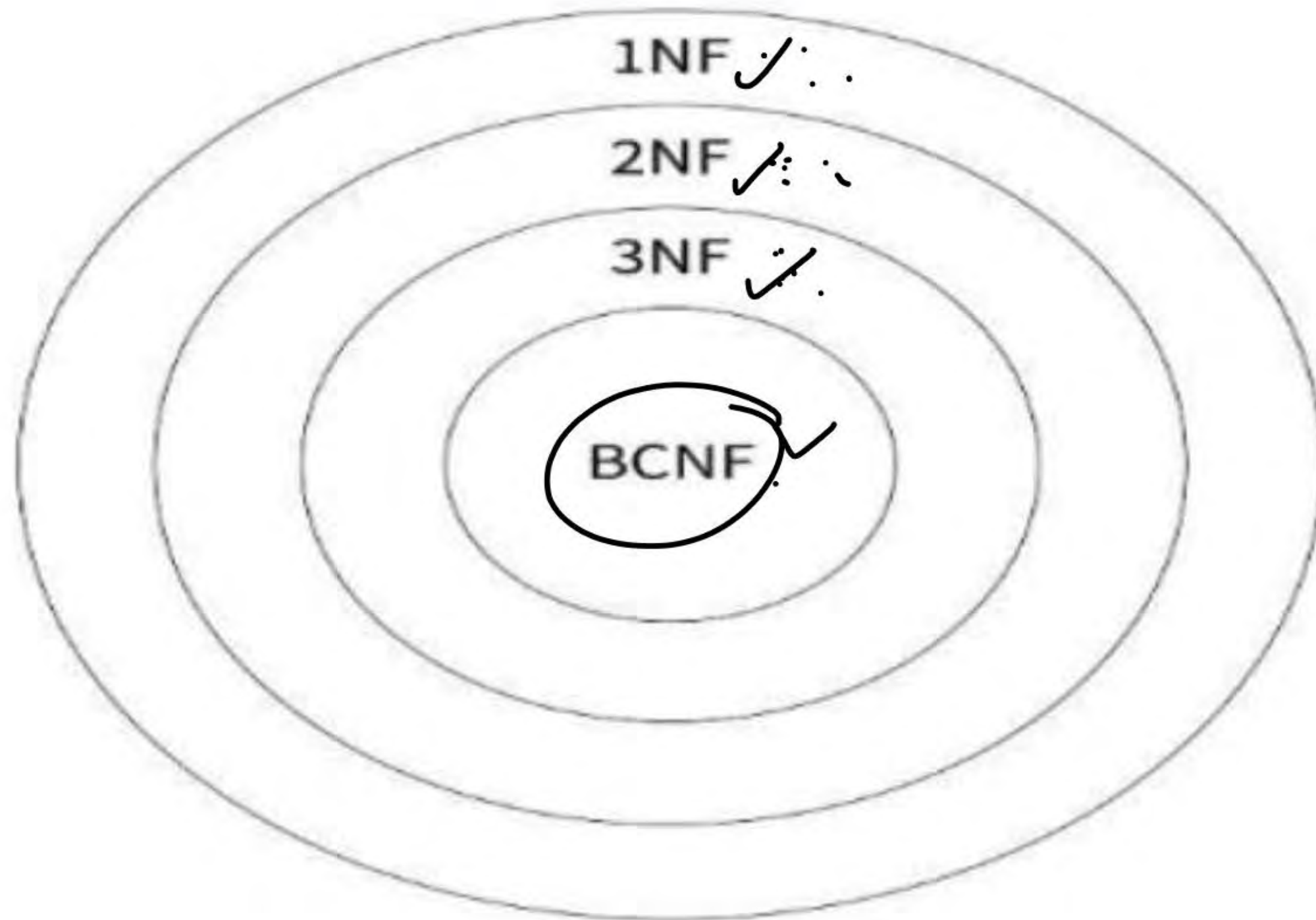
VOTER_ID → AGE

VOTER_ID → ROLL_NO }

Note:- L.H.S Should be function of Candidate or Super key



Topic : Normal Forms





Topic : Normal Forms

1 st - Normal form	2 nd - Normal Form	3 rd - Normal Form	BCNF
- No Multivalued <u>Attribute</u> - Only Singled Valued Partial Depend $D \xrightarrow{PA} A$ PA (K: $\rightarrow AB$ $PA \rightarrow (A, B)$ $NPA \rightarrow (C, D, E)$ $A \rightarrow C$ NPA	- In 1st NF - No Partial <u>Dependency</u> Suppose Candidate key: $AB, \{B \rightarrow C\}$ Formula to check Partial Dependency $\rightarrow$ L.H.S Should be Proper subset of Candidate key and R.H.S should be <u>Non-prime attribute</u> - Only Full <u>Functional Dependency</u> FK: AB $A \rightarrow C, BD \rightarrow C$	- In 2nd NF - No Transitive Dependency - No <u>Non-Prime</u> should Determine <u>Non Prime</u> - Rule to check 3-NF = L.H.S Must be a Candidate Key or Super key Or R.H.S is a prime Attribute	- In 3rd NF - L.H.S must be <u>candidate key or super key</u>



Topic : Normal Forms

Question:- $R(ABCDEF)$, Check the Highest Normal Form

$AB \rightarrow \underline{C}, C \rightarrow DE, E \rightarrow F, F \rightarrow A$

$\underline{AB}^+ = \{A, B, C, D, E, F\}$

BCNF

Diagram illustrating the closure of $\underline{AB}$:

$\underline{AB}$ (circled) leads to F, B , then E, B , then C, D .



Topic : Normal Forms

Question:- R(ABCDEF), Check the Highest Normal Form

$AB \rightarrow C, C \rightarrow DE, E \rightarrow F, F \rightarrow A$

Step1:- Find all candidate Key

CK: {AB, FB, EB, CB}

Step2:- Write down all prime attribute

Prime Attribute:- {A, B, C, E, F}

Step3:- Write down all Non Prime attribute

Non-Prime Attribute:- {D}

Partial Dependency:- {L.H.S is proper subset of ck or sk

And R.H.S Non-prime attribute}

~~BCNF:-~~

~~3rd NF:-~~

2nd NF:-

1st NF

$AB \rightarrow C$

Yes

Yes

yes

Yes

$C \rightarrow DE$

No

No

NO

Yes

$E \rightarrow F$

No

Yes

Yes

Yes

$F \rightarrow A$

No

yes

Yes

Yes





Topic : Cartesian Product

The Cartesian product in SQL is the result of combining every row from one table with every row from another. This operation is most often generated through a CROSS JOIN, forming a result set containing all possible pairs of rows from the two tables. While powerful, it can lead to very large result sets, especially when working with tables containing many rows, so it should be used with caution. In fact, I'll bet many developers probably first discover Cartesian products the hard way by accidentally omitting a join condition of some kind, only to end up with a massive result set.



Topic : Cartesian Product



TEACHING
WALLAH

Degree:- Total Number Column ✓

Cardinality:- Total Number Of Tuples ✓

There is 2 Relation R1 and R2 so Cartesian Product is R1XR2

It combines R1 and R2 without any condition. It is denoted by X.

Degree of R1 XR2 = degree of R1 + degree of R2

{degree = total no of columns}

Cardinality of R1 X R2 = ³Cardinality of R1 X ⁵Cardinality of R2 = 15

{Cardinality = multiply of total tuple in Relation R1 & R2}



Topic : Cartesian Product

R1 TABLE

ROLL	NAME
1	ROHAN SHARMA
2	RAHUL GUPTA
3	HARI OM

R2 TABLE

SALES YEAR	TARGET ACHEIVE
2024	4025
2025	7036

$$\textcircled{2} \times \textcircled{3}$$

$$\textcircled{2} \times \textcircled{2}$$

R1 X R2 TABLE

$$\text{DEGREE} \Rightarrow \underline{2+2=4}$$

$$\text{CARDENALITY} \Rightarrow 3 \times 2 = 6$$

ROLL NO	NAME	SALES YEAR	TARGET ACHEIVE
1	ROHAN SHARMA	2024	4025
1	ROHAN SHARMA	2025	7036
2	RAHUL GUPTA	2024	4025
2	RAHUL GUPTA	2025	7036
3	HARI OM	2024	4025
3	HARI OM	2025	7036



Topic : Cartesian Product

Table : Customers

Customers

ID	Name
1	Raj Mehta
2	Sanjay Mishra
3	Aditi Gupta

Table_Name : Shopping_Details

Shopping

ID	Item_Name
2	Chips
3	Chocolate

Table : Result

3 x 2 = 6 rows

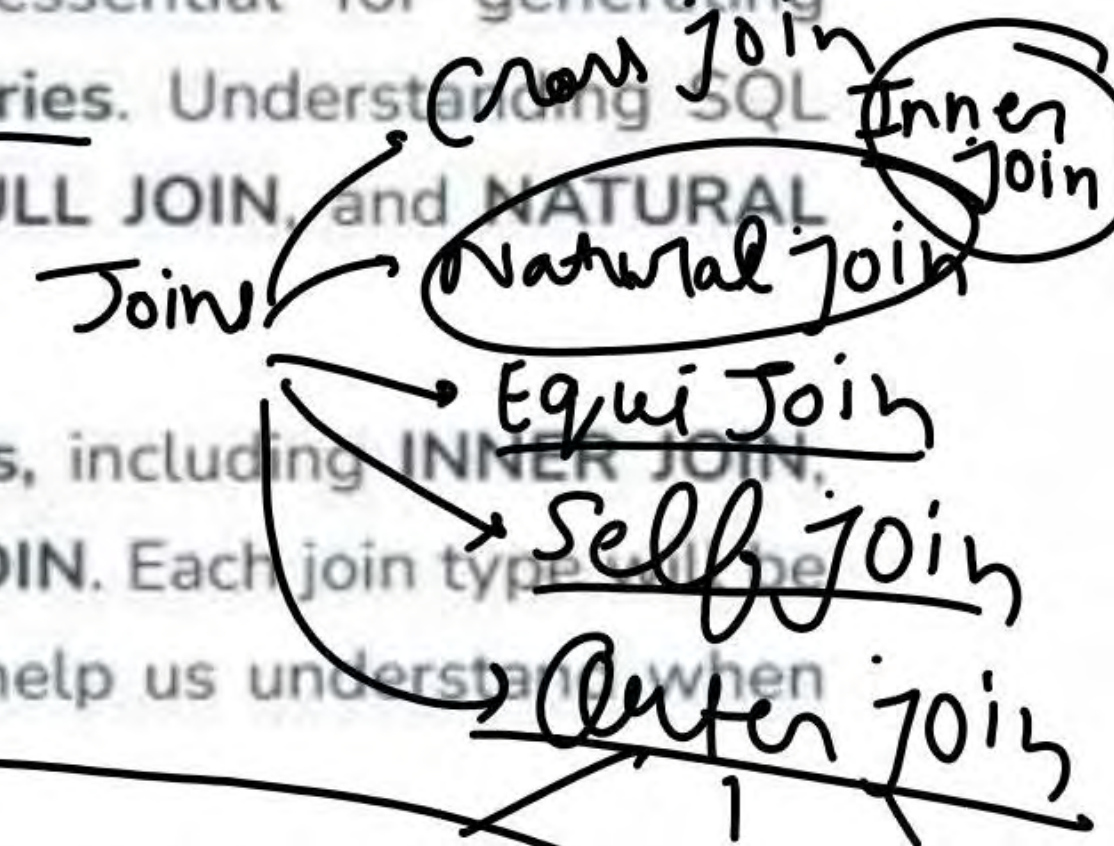
Customers CustID	Name	Shopping_d etails.CustID	Item_Name
1	Raj Mehta	2	Chips
1 ,	Raj Mehta	3	Chocolate
2	Sanjay Mishra	2	Chips
2 ,	Sanjay Mishra	3	Chocolate
3	Aditi Gupta	2	Chips
3 ,	Aditi Gupta	3	Chocolate



Topic : SQL JOINS

SQL joins are fundamental tools for combining data from multiple tables in relational databases. Joins allow efficient data retrieval, which is essential for generating meaningful observations and solving complex business queries. Understanding SQL join types, such as INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL JOIN, and NATURAL JOIN, is critical for working with relational databases.

In this article, we will cover the different types of SQL joins, including INNER JOIN, LEFT OUTER JOIN, RIGHT JOIN, FULL JOIN, and NATURAL JOIN. Each join type will be explained with examples, syntax, and practical use cases to help us understand when and how to use these joins effectively.



- SQL JOINS = CROSS PRODUCT + SELECT STATEMENT (CONDITION)
- Cross Product is also known as Cartesian Product.



Topic : SQL JOINS

What is SQL Join?

An SQL JOIN clause is used to **query and access data** from multiple tables by establishing **logical relationships** between them. It can access data from multiple tables simultaneously using common key values shared across different tables. We can use **SQL JOIN** with **multiple tables**. It can also be paired with other clauses, the most popular use will be using JOIN with WHERE clause to filter data retrieval.

Before diving into the specifics, let's visualize how each join type operates:

- **INNER JOIN**: Returns only the rows where there is a match in both tables.
- **LEFT JOIN (LEFT OUTER JOIN)**: Returns all rows from the left table, and the matched rows from the right table. If there's no match, NULL values are returned for columns from the right table.
- **RIGHT JOIN (RIGHT OUTER JOIN)**: Returns all rows from the right table, and the matched rows from the left table. If there's no match, NULL values are returned for columns from the left table.
- **FULL JOIN (FULL OUTER JOIN)**: Returns all rows when there is a match in one of the tables. If there's no match, NULL values are returned for columns from the table without a match.



Topic : NATURAL JOINS

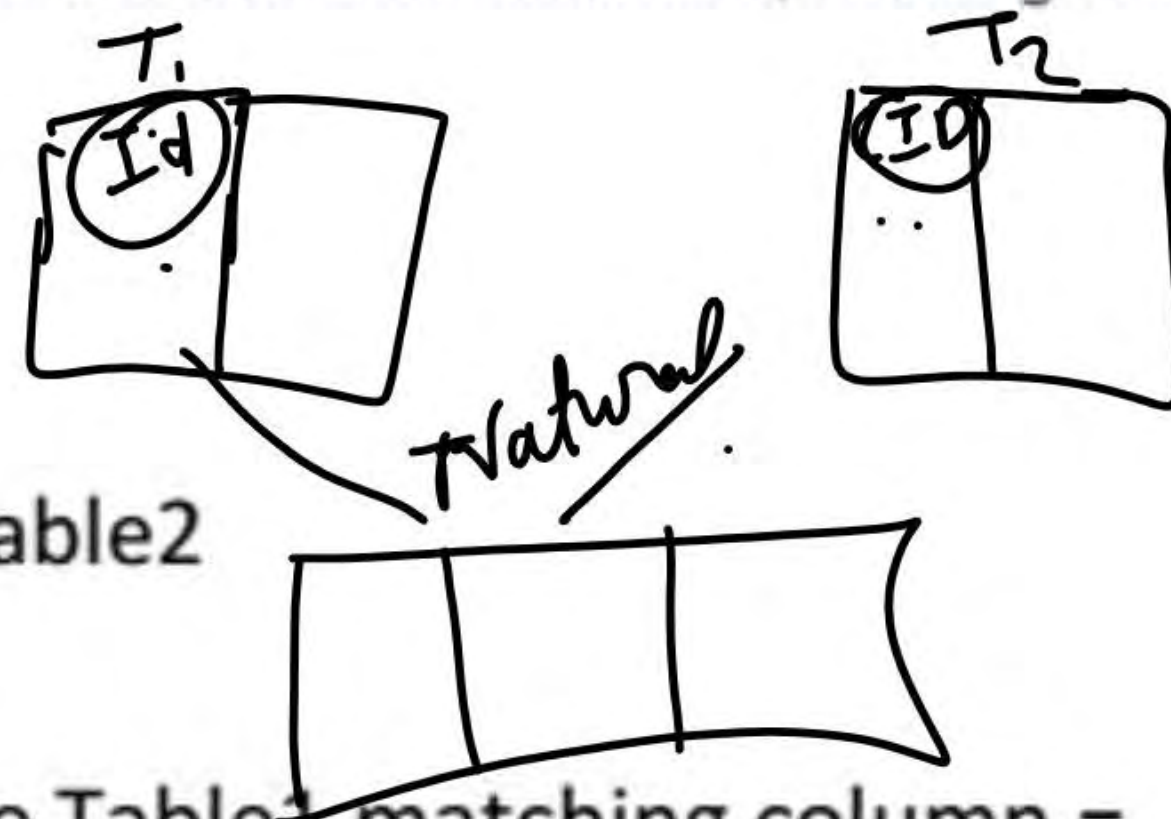
A **Natural Join** performs a join between two tables based on columns that have the same name and compatible data types. This join operation automatically determines the common columns between the tables and removes duplicates in the result set. The resulting table will contain only one copy of the common columns, making the result concise and free from redundant data.

SYNTAX:-

Select * from Table1 Natural Joins Table2

Or

Select * From TABLE1, TABLE2 Where Table1.matching column = Table2.matching Column.





Topic : NATURAL JOINS

Natural Joins:-

Find the Emp Names who is working in a department.

PK

E_NO ✓	E_NAME ✓	ADDRESS
1	RAM	<u>DELHI</u>
2	VARUN	MUMBAI
3	RAVI	KANPUR
4	AMRIT	DELHI

PK

DEPT_NO	NAME ✓	E_NO ✓
D1	HR	1
D2	IT	2 //
D3	MARKETING	3 //

FK

Select E_NAME From Employee Natural Joins Department where Employee.E_NO = Department.E_NO

**Topic : NATURAL JOINS****CARTESIAN PRODUCT:-**

HIGHLIGHTED SHOW THE RESULT

E_NO	E_NAME	DEPT_NO	E_NO
1	RAM	D1	1
1	RAM	D2	2
1	RAM	D3	4
2	VARUN	D1	1
2	VARUN	D2	2
2	VARUN	D3	4
3	RAVI	D1	1
3	RAVI	D2	2
3	RAVI	D3	4
4	AMRIT	D1	1
4	AMRIT	D2	2
4	AMRIT	D3	4



Topic : INNER JOINS

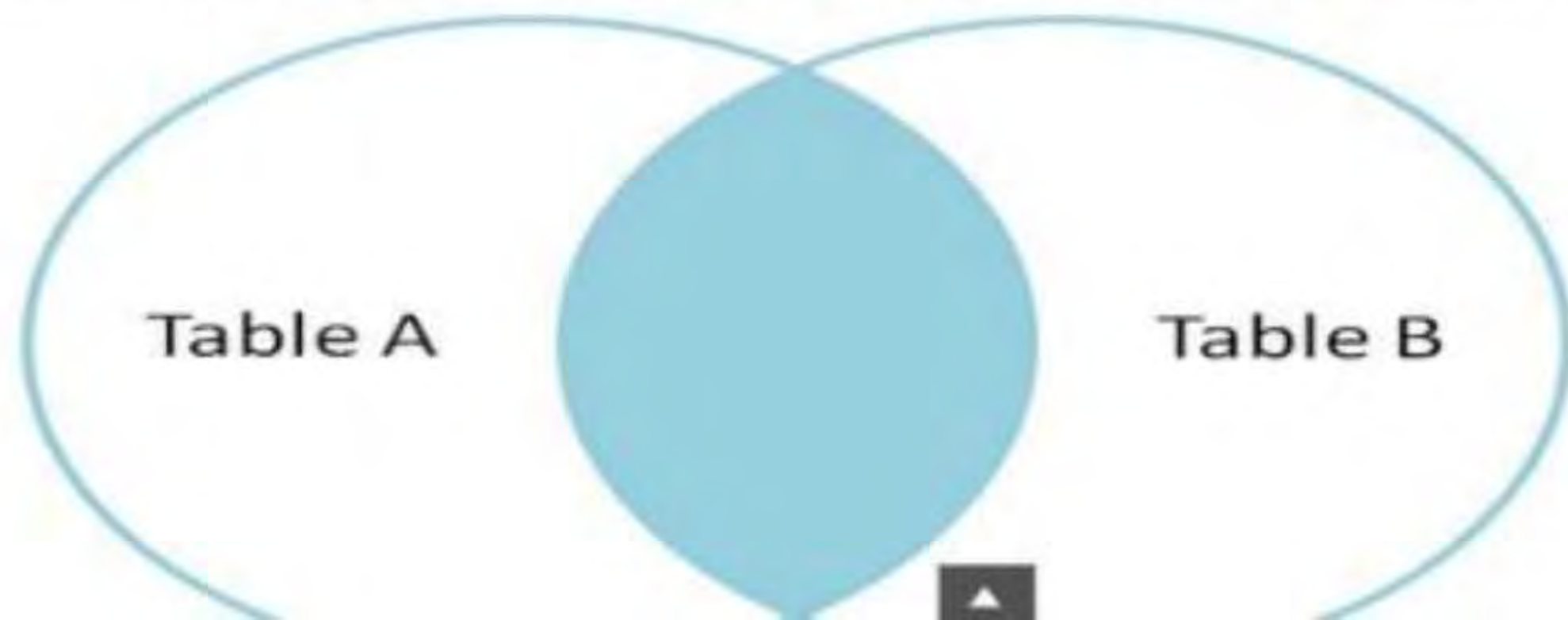
1. SQL INNER JOIN

The INNER JOIN keyword selects all rows from both the tables as long as the condition is satisfied. This keyword will create the **result-set** by combining all rows from both the tables where the **condition satisfies** i.e value of the common field will be the same.

Syntax

```
SELECT table1.column1,table1.column2,table2.column1,.... FROM table1 INNER  
JOIN table2 ON table1.matching_column = table2.matching_column;
```

Note: We can also write JOIN instead of INNER JOIN. JOIN is same as INNER JOIN.





Topic : INNER JOINS

Table_Name : Customers

CustID	Name	Phone_Number
1	Raj Mehra	98540XXXXX
2	Sanjay Mishra	88888XXXXX
3	Aditi Gupta	67809XXXXX
4	Manish Chopra	12345XXXXX

Table_Name : Shopping_Details

ItemID	CustID	Item_Name	Quantity
1	2	Chips	2
2	3	Chocolate	5
3	5	Dress	8



Temporary Table

Customers CustID	Name	Phone_Number	Item_ID	Shopping_d etails,CustID	Item_Name	Quantity
1	Raj Mehra	98540XXXXX	NULL	NULL	NULL	NULL
2	Sanjay Mishra	88888XXXXX	1	2	Chips	2
3	Aditi Gupta	67809XXXXX	2	3	Chocolate	5
4	Manish Chopra	12345XXXXX	NULL	NULL	NULL	NULL
NULL	NULL	NULL	3	5	Dress	8

Resultant Table

Name	Item_Name	Quantity
Sanjay Mishra	Chips	2
Aditi Gupta	Chocolate	5

**Topic : Difference between INNER JOINS and NATURAL JOINS**

Difference Between InnerJoin & NaturalJoin

Following are the key differences between InnerJoin and NaturalJoin -

Criteria	Inner Join	Natural Join
Join Condition	Requires an explicit condition using the ON clause.	Automatically matches columns with the same name.
Column Selection	Requires the user to Specify which columns to compare.	Automatically detects columns with the same name.
Duplicate Columns	May return duplicate columns In the result set.	Does not return duplicate columns.
Control	Offers more control over the Join Operation.	Less control, as it depends on column names.
Risk Errors	Less risk if columns are explicitly specified.	Higher risk if unintended columns share the same name.



Topic : EQUI JOINS

1. EQUI JOIN :

EQUI JOIN creates a JOIN for equality or matching column(s) values of the relative tables. EQUI JOIN also create JOIN by using JOIN with ON and then providing the names of the columns with their relative tables to check equality using equal sign (=).

Syntax :

```
SELECT column_list  
FROM table1, table2....  
WHERE table1.column_name =  
table2.column_name;
```




Topic : EQUI JOINS

Equi Joins:-

Find the Emp Names who is working in a department having same location as their address.

E_NO	E_NAME	ADDRESS
1	RAM	DELHI
2	VARUN	MUMBAI
3	RAVI	KANPUR
4	AMRIT	DELHI

DEPT_NO	LOCATION	E_NO
D1	DELHI	1
D2	PUNE	2
D3	PATANA	3

Select E_name from Employee ,Department where Employee.E_NO = Department.E.NO and
Employee.Address=Department.Location

Note:- Here Equi Joins looking like Natural Join along with equal Condition.



Topic : EQUI JOINS

Equi Joins:-

Find the Emp Names who is working in a department having same location as their address.

E_NO	E_NAME	ADDRESS
1	RAM	DELHI
2	VARUN	MUMBAI
3	RAVI	KANPUR
4	AMRIT	DELHI

DEPT_NO	LOCATION	E_NO
D1	DELHI	1
D2	PUNE	2
D3	PATANA	3

Select E_name from Employee ,Department where Employee.E_NO = Department.E.NO and
Employee.Address=Department.Location

Note:- Here Equi Joins looking like Natural Join along with equal Condition.



Topic : EQUI JOINS

Highlighted show the output of Equi joins



E_NO	E_NAME	DEPT_NO	E_NO
1	RAM	DELHI	1
1	RAM	PUNE	2
1	RAM	PATANA	3
2	VARUN	DELHI	1
2	VARUN	PUNE	2
2	VARUN	PATANA	3
3	RAVI	DELHI	1
3	RAVI	PUNE	2
3	RAVI	PATANA	3
4	AMRIT	DELHI	1
4	AMRIT	PUNE	2
4	AMRIT	PATANA	3

THANK YOU